ComeBack — Project Explainer & Interview Guide

> ***ComeBack*** *— a privacy-first habit and workout tracker with an AI coach, where a lapse is treated as a comeback to rebuild, not a failure that resets you to zero. Repository:* `prathamshukla28/-ComeBack-` *· License: MIT · Author: Pratham Shukla*

Part 1 — What Is ComeBack?

The one-line pitch

ComeBack is a cross-platform mobile app (iOS, Android, Web via Expo) that lets you log workouts and hard-to-break habits, tracks **relapse-aware streaks** that reframe a slip as a rebuild, and gives you an **AI coach** grounded in your own history — all on top of a database where one user can never see another's data.

The problem it solves

Most habit trackers punish you: miss a day and your streak cold-resets to zero, which is exactly when people quit. ComeBack's thesis is recovery *after* a lapse. A relapse never zeroes you — it measures the broken run against your personal best and nudges you to beat it.

At the same time, generic AI chatbots don't know your numbers, and dumping a user's whole history into every prompt is slow, expensive, and leaky. ComeBack adds a retrieval layer so the coach answers from *relevant* curated knowledge plus *this user's* own logged history — with data isolation enforced by the database, not by hopeful application code.

Who it's for

People rebuilding a habit — cutting down cigarettes or alcohol, staying consistent with training — who want an honest, private companion. The journal tracks addiction-adjacent and intimate data, so privacy and a crisis-safe coach are first-class features, not afterthoughts.

Part 2 — What It Does (features & flows)

ComeBack is a bottom-tab app. Each tab is a file-based route under `app/(tabs)/`:

- **Today** (`index.tsx`) — one-glance dashboard of today's sets, streaks, and habits.
- **Workout** (`workout.tsx`) — log sets, reps, weight, RIR; detects new estimated-1RM PRs.
- **Habits** (`habits.tsx`) — journal cigarettes, alcohol, intimacy with clean-streak counters.
- **Intimacy** (`intimacy.tsx`) — a private tab behind a Face ID / PIN gate.
- **Guru** (`guru.tsx`) — a no-BS fitness coach; routes through RAG when configured.
- **Coach** (`coach.tsx`) — a warm life coach that remembers you (Pro-gated).
- **Trends** (`trends.tsx`) — week-over-week deltas and sparklines, coloured toward your goal.
- **Settings** (`settings.tsx`) — profile, Gemini API key, preferences.

Key features

- Relapse-aware streaks: current clean run, personal best, and the length of a run a relapse

just broke, so the UI can say "you've done longer — rebuild."

- Offline-safe logging: a tap on a flaky connection is persisted to a durable outbox and

replayed on reconnect; logs are never lost and never double-counted.

- AI coach with optional RAG grounding over a curated exercise KB plus your own history.
- A coach safety net: self-harm signals short-circuit the LLM and surface vetted crisis lines.
- Biometric lock with a salted-hashed, rate-limited PIN fallback for the private tab.
- RevenueCat subscriptions behind a single `pro` entitlement.

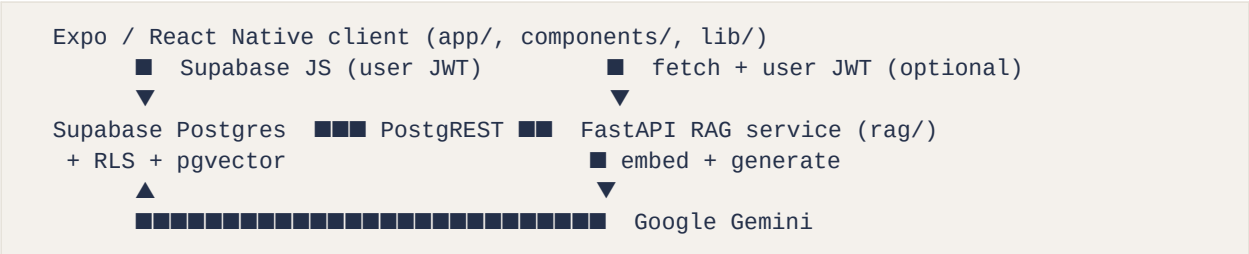
A concrete example

You smoked yesterday after 12 clean days. You open Habits and tap "log cigarette." You're on the subway with no signal, so the write can't reach Supabase — instead of silently failing, the tap is saved to the on-device outbox and replays automatically when you resurface. The streak card doesn't shame you with a `0`: it shows the 12-day run you just broke against your all-time best and frames the next clean day as day one of the rebuild. Later you ask the Guru "how do I come back from a deload week?" — it answers from the curated knowledge base plus *your* logged sets, never anyone else's.

Part 3 — Architecture

ComeBack has two halves: a **TypeScript Expo client** and an **optional Python RAG service**, with **Supabase Postgres** as the shared, security-bearing core. The app is designed to build and run fully even when the RAG service, billing, and AI key are all absent.

The layers



Layer	Where	Responsibility
Routing / screens	`app/`	expo-router file-based tabs, auth, modal, paywall.
UI primitives	`components/`	`Screen`, `Card`, `Button`, `Chat`, `AnimatedNumber`, `Confetti`.
Client logic	`lib/`	Supabase access, streaks, safety, PIN, outbox, subscription, AI.
Data store	`db/schema.sql`	Tables, owner_id trigger, owner-only RLS, seed exercises.

Layer	Where	Responsibility
Vector store	`db/rag.sql`	pgvector tables + `SECURITY INVOKER` match/upsert RPCs.
RAG service	`rag/`	FastAPI: verify token, retrieve, ground, call Gemini.
Design tokens	`constants/`	Colour palettes for light/dark theming.

Client state and data

Server data flows through **TanStack Query** (`lib/queries.ts`) against a single Supabase client (`lib/supabase.ts`); UI-local state uses React state and Zustand. Every row carries an `owner_id` that a trigger auto-fills from `auth.uid()`, and **row-level security** restricts every table to its owner. The client never holds a privileged key — it talks to Postgres as the signed-in user.

The AI coach, two paths

The `Chat` component (`components/Chat.tsx`) is engine-agnostic: a screen either lets it call Gemini directly (`lib/gemini.ts`, dumping a 30-day context block) or supplies an `ask` override that routes through the RAG service (`lib/rag.ts`). The Guru tab uses RAG when `EXPO_PUBLIC_RAG_URL` is set and falls back to the direct path otherwise — the same app binary works both ways.

Graceful degradation everywhere

No Gemini key → the chat tells you to paste one in Settings. No RAG URL → direct Gemini. No RevenueCat key or native build (web, Expo Go, tests) → billing is simply off and Pro stays locked. No biometric hardware → the PIN fallback. Nothing hard-crashes on a missing optional.

Part 4 — The RAG Service & the Data-Isolation Guarantee

The most defensible subsystem is the retrieval-augmented coach in `rag/` and, above all, how it keeps one user's history invisible to another. The claim the project stands behind: *a user's coach can never retrieve another user's data, even if the RAG service has a bug.*

The pipeline

A `/rag/coach` call embeds the question with Gemini `text-embedding-004` (768 dims) as a `RETRIEVAL_QUERY`, then fans out concurrently to two pgvector stores, filters by a similarity threshold, assembles a grounded prompt with labelled `[KB#]` / `[H#]` sources, and asks `gemini-2.0-flash` for the answer. `/rag/retrieve` returns the same context without generating, for transparency and debugging.

Store	Scope	RLS	Written by
`exercise_kb`	shared, curated	`select` for all; no client writes	offline ingest (service-role)
`user_memories`	per user	`owner_id = auth.uid()` for all ops	the user, via `/rag/memories`

Why it's safe — layered enforcement

- **RLS is the invariant.** `user_memories` policies use `owner_id = auth.uid()`. Postgres

filters rows by the authenticated user regardless of what a query asks for.

- **The user's JWT is forwarded on every user-data call.** The service passes the end user's

Supabase access token straight to PostgREST (`Authorization: Bearer <user-jwt>`), so `auth.uid()` inside the `SECURITY INVOKER` RPC *is* the end user. The service holds no privileged key on the request path, so there is no capability to read across users.

- **Verify before you spend.** `require_user` runs ahead of every Gemini call. With

`SUPABASE_JWT_SECRET` set it is a local HS256 check; without it the service asks Supabase Auth (`GET /auth/v1/user`), covering projects on asymmetric signing keys. A bad token is a `401` that costs nothing; an unreachable verifier is a `503`, never an assumed pass.

- **Per-user throttle.** After verification, a sliding window caps how fast one subject can run

up the Gemini bill; over the limit is `429` with `Retry-After`, and it has spent nothing.

The alternative — a service-role key plus `where user_id = ?` — makes isolation depend on every query remembering the filter; one missed `WHERE` leaks everything. Here the filter is a database invariant. The service-role key exists only for the offline KB ingest, which reads no user rows.

Retrieval-quality choices

Stored chunks are embedded as `RETRIEVAL_DOCUMENT` and queries as `RETRIEVAL_QUERY` (task-typed embeddings align better than embedding both the same way). Cosine distance (`<=>`) with an `ivfflat` index drives ranking, `similarity = 1 - distance` is thresholded so weak matches don't pad the prompt, and user history is stored as weekly summaries with a stable `dedupe_key` so re-ingesting a rolling window upserts instead of piling up near-duplicates.

Tests that pin it

`tests/test_api.py` asserts the outbound memory lookup carries the **user's** bearer token, not the anon/service key; `tests/test_auth_gate.py` asserts a junk token never reaches Gemini; `tests/test_ratelimit.py` asserts a `429` spends nothing. All network is mocked via `respx`.

Part 5 — Relapse-Aware Streaks, Offline Logging & Coach Safety

Three client subsystems carry the product's thesis, and each is written as pure, unit-testable functions so the hard logic is provable without a device or a network.

Relapse-aware streak math (`lib/streaks.ts`)

`computeStreakStats` takes unsorted `YYYY-MM-DD` occurrence days and returns the current clean run, the longest ever, the `previousBest` (longest *completed* run — the record to chase), whether the user `relapsedToday`, and `brokenStreak` (the length of the run a today-relapse just broke). It works on integer epoch-day indices so it is timezone-free. A future-dated log clamps to `0` rather than reporting a negative streak. This is what lets the UI say "you've done longer — rebuild" instead of silently resetting to zero.

Durable offline outbox (`lib/outbox.ts`)

The app is online-only against Supabase, so a tap on a flaky connection used to be thrown away. Now `logHabitResilient` mints a client-side UUID, tries the live insert, and on a **network** failure persists the op to an AsyncStorage outbox; genuine server rejections (RLS/auth) still throw so the caller sees them. Replay is **idempotent**: the client UUID is also the row's primary key, so an insert that committed but whose response was lost in transit replays into a `23505` unique-violation — treated as "already landed" — instead of double-logging. Replay stops at the first network error and drops ops the server rejects outright, so a poison write can't wedge the queue forever. The queue transforms (`addOp`, `removeOp`, `parseQueue`, `toRow`) are pure.

Coach safety net (`lib/safety.ts`)

Because the journal holds addiction data and the coach is a confidant, a crisis response is a feature. `detectCrisis` is a pure, offline, deliberately high-recall regex check: in the `Chat` send flow, a self-harm signal **short-circuits the LLM entirely** and replies with vetted human resources (988, Crisis Text Line, Samaritans, findahelpline) — the message never leaves the phone to a model. As belt-and-suspenders, a `SAFETY_PREAMBLE` is appended to every coach/guru system prompt to keep the model in bounds for anything a regex misses.

PIN hardening (`lib/pin.ts`)

The private Intimacy tab is gated by biometrics with a PIN fallback. The PIN is never stored in plaintext — it is a salted SHA-256 digest in `expo-secure-store` (OS keychain, `WHEN_UNLOCKED_THIS_DEVICE_ONLY`). `computeLockout` is a pure escalating-backoff policy: five free attempts, then a lockout that doubles each failure (30s → 60s → 120s …) capped near an hour. A legacy plaintext PIN from older builds is migrated to a salted hash on first successful verify.

Subscriptions (`lib/purchases.ts`, `lib/subscription.tsx`)

RevenueCat wraps store billing behind a single `pro` entitlement. The native SDK is loaded lazily inside a `try/catch`, so on web, in Expo Go, and under Jest it is simply absent and Pro stays `false` without crashing. `hasProEntitlement` is a pure predicate so it's unit-testable without the native module; `SubscriptionProvider` exposes `useIsPro()` / `useSubscription()` and the Coach tab renders `<Paywall>` when the user isn't Pro.

Part 6 — Tech Stack

Area	Choice
Runtime	Expo SDK 54, React Native 0.81, React 19, Node ≥ 22
Language	TypeScript 5.9 (strict); Python 3.11 for `rag/`
Routing	expo-router (file-based)
State / data	Zustand, TanStack Query
Data store	Supabase Postgres + RLS; pgvector for RAG
AI	Gemini `gemini-2.0-flash` + `text-embedding-004` (768 dims)
RAG service	FastAPI, httpx, PyJWT, respx (tests)

Area	Choice
Billing	RevenueCat (`react-native-purchases`), single `pro` entitlement
Animation	react-native-reanimated v4 + Worklets
Secure storage	expo-secure-store, expo-crypto, AsyncStorage
Validation	zod-validated env (`lib/env.ts`)
Testing	Jest + Testing Library (client); pytest (service)
CI / quality	GitHub Actions, CodeQL (JS/TS + Python), Dependabot, release-please
Local hooks	Husky + lint-staged + commitlint (Conventional Commits)

Part 7 — Honest Limitations

A mature engineer names the trade-offs:

- The RAG rate limiter is **in-process** (one sliding window per uvicorn worker), so the

effective ceiling is `limit x workers`; a shared store like Redis is the upgrade path if the service is ever scaled out (logged as SPEC-GAP-016).

- The embedding dimension (768) is **hard-coded** across `db/rag.sql`, `config.py`, and

validated per embedding. Changing the model means a migration plus a full re-ingest.

- `ivfflat` lists are tuned for a **small** curated KB; a large KB would want `hnsw`.
- The direct Gemini path dumps a **30-day context block** into every prompt — cheaper to build

than RAG but slower, costlier, and less precise; RAG is the better path but optional.

- The crisis detector is a **regex**, chosen for high recall on-device; it will have false

positives (an extra set of hotline numbers) and can miss novel phrasing — the prompt-level safety preamble is the backstop.

- The Supabase schema is multi-user-safe but the product is framed around a **single user**;

the AI "memory extractor" writes free-form facts the user can't currently curate in-app.

- Billing, RAG, and AI are all **optional** by design, which keeps the app runnable everywhere

but means a full-featured deployment needs several external services wired up (see `docs/SETUP.md`, `docs/MONETIZATION.md`).

Part 8 — Interview Questions & Answers

A. Project overview

Q1. Give me the 30-second overview of ComeBack. ComeBack is a privacy-first habit and workout tracker with an AI coach, built with Expo / React Native and Supabase and running on iOS, Android, and Web. Its thesis is recovery after a lapse: relapse-aware streaks never cold-reset you to zero, they reframe the broken run against your best and nudge a rebuild. The coach is retrieval-augmented — grounded in a curated exercise KB plus your own history in pgvector — and user data is isolated at the database with row-level security, not by application code.

Q2. Why did you build it? What gap does it fill? Most trackers punish the exact moment people quit — the broken streak — by resetting to zero. And generic chatbots don't know your numbers, while dumping your whole history into every prompt is slow, expensive, and leak-prone. ComeBack reframes the lapse as a comeback and adds a retrieval layer so the coach answers from *relevant* context about *this* user, with isolation the database guarantees.

Q3. Where did the name come from? "ComeBack" is the product thesis in one word: a lapse is a comeback to rebuild, not a failure that erases your progress. The streak engine is literally built so a relapse shows you the run you broke and your record to beat, instead of a silent reset.

B. Architecture & design

Q4. Walk me through the architecture. Two halves over a shared core. The Expo/TypeScript client (`app/`, `components/`, `lib/`) talks to Supabase Postgres as the signed-in user; TanStack Query manages server state. An optional Python FastAPI service (`rag/`) does retrieval-augmented generation. Supabase is the security-bearing core: every row has an `owner_id` and owner-only RLS, and pgvector lives in the same database so vectors inherit the same isolation. The app is built to run fully with the RAG service, billing, and AI key all absent.

Q5. Why put the RAG service in Python when the app is TypeScript? It's where the ecosystem and the clearest security story are. FastAPI plus httpx plus PyJWT made the verify-then-spend gate and JWT forwarding straightforward, and respx let me mock every outbound call so the isolation contract is tested without real network. Keeping it a separate, optional service also means the mobile app has zero new attack surface when it's not deployed.

Q6. How does the `Chat` component stay engine-agnostic? `Chat` takes a config with a `systemPrompt` and an optional `ask` override. If `ask` is present it produces the reply (used to route through the RAG service); otherwise `Chat` calls Gemini directly with the system prompt and history. So the Guru tab flips between RAG and direct Gemini purely by whether `EXPO_PUBLIC_RAG_URL` is set, with no change to the chat UI.

Q7. How does the app degrade when optional services are missing? By design, nothing optional can hard-crash it. No Gemini key prompts the user to add one; no RAG URL falls back to direct Gemini; no RevenueCat key or native build means billing is off and Pro stays locked; no biometric hardware means the PIN fallback. The RevenueCat SDK is even loaded lazily in a `try/catch` so its absence on web or under Jest is a no-op.

C. Domain (mobile / RAG / product)

Q8. Explain the data-isolation guarantee and why it's a database invariant. Every user-data query runs under the end user's Supabase JWT, forwarded all the way to PostgREST, and the retrieval RPCs are `SECURITY INVOKER` so they execute with the caller's privileges. `user_memories` has an RLS policy `owner_id = auth.uid()`, so Postgres filters rows by the authenticated user no matter what the query asks. The RAG service holds no privileged key on the request path, so even a bug can't read across

users. That's the difference from a service-role key plus a `WHERE user_id = ?` filter, where one forgotten clause leaks everything.

Q9. Why verify the token before any retrieval or generation? Because every authenticated request spends real money — a Gemini embedding and usually a generation, billed to the service's key. `require_user` runs first: a bad token is a `401` that costs nothing, an unreachable verifier is a `503` (never an assumed pass), and only after verification does the per-user rate limiter take a slot. So an anonymous flood is rejected before it's counted against anyone and before a cent is spent.

Q10. Why task-typed embeddings and a similarity threshold? Gemini's `text-embedding-004` supports task types; embedding stored chunks as `RETRIEVAL_DOCUMENT` and the query as `RETRIEVAL_QUERY` aligns them into a shared space and measurably beats embedding both the same way. The threshold (`similarity = 1 - cosine distance`) drops weak matches so the grounded prompt isn't padded with irrelevant noise that would dilute the answer and waste tokens.

Q11. How do you keep the coach safe around self-harm? Two layers. `detectCrisis` is a pure, offline, high-recall regex that runs before anything leaves the phone; a positive match short-circuits the LLM entirely and returns vetted crisis resources. On top of that, a `SAFETY_PREAMBLE` is appended to every system prompt so the model stays in bounds for phrasing the regex misses. A false positive just costs an extra set of hotline numbers; the false negative is the failure mode I refuse to accept.

D. Deep technical

Q12. How does the offline outbox avoid losing or double-counting a log? `logHabitResilient` mints a client-side UUID and tries the live insert. A network failure queues the op to AsyncStorage; a real server rejection still throws. The key trick is idempotency: the client UUID is also the row's primary key, so if an insert committed server-side but the device never got the response, the replay hits a `23505` unique-violation, which I treat as "already landed" rather than inserting a duplicate. Replay stops at the first network error and drops server-rejected ops so a poison write can't wedge the queue.

Q13. Walk through the relapse-aware streak computation. `computeStreakStats` dedupes and sorts occurrence days into integer epoch-day indices, so it's timezone-free. Completed clean runs are the gaps strictly between consecutive occurrences; the current run is today minus the last occurrence, clamped to zero so a future-dated log can't go negative. `previousBest` is the max completed gap — the record to chase — and if the last occurrence is today with at least one prior, `relapsedToday` is true and `brokenStreak` is the run that relapse just broke. That's the data the "you've done longer, rebuild" UI needs.

Q14. How is the PIN stored and brute-force protected? SecureStore already encrypts with the OS keychain, but a 4-digit PIN is guessable in seconds, so it's never stored or compared in plaintext. `setPin` writes `salt:SHA-256(salt:pin)` and verify compares digests. `computeLockout` is a pure escalating backoff: five free attempts, then a lockout that doubles each further failure, capped near an hour. Keeping the policy pure means I unit-test the lockout math without touching native crypto, and a legacy plaintext PIN is upgraded to a salted hash on first successful verify.

Q15. Why is the Gemini key not an environment variable? `EXPO_PUBLIC_*` values are inlined into the client bundle in plaintext, so a key there would ship to every user. Instead each user pastes their own Gemini key in Settings and it's kept in the OS keychain via `expo-secure-store`. The env schema (`lib/env.ts`, zod-validated) only holds things that are safe to bundle — the Supabase URL and anon key, the optional RAG URL, and the public RevenueCat SDK keys.

Q16. How does the RAG rate limiter work and what's its honest limitation? It's a sliding window keyed by the token subject, kept in process memory with a deque of hit timestamps per user and periodic pruning so memory stays bounded. `limit <= 0` disables it and `now` is injectable so the policy is testable without sleeping. The honest limitation: with several uvicorn workers each keeps its own counts, so the real ceiling is `limit x workers`. A shared store like Redis is the documented upgrade path (SPEC-GAP-016).

Q17. How do the two vector stores differ at the SQL level? `exercise_kb` is shared curated knowledge: `select` for everyone, no client writes, seeded by an offline service-role ingest. `user_memories` is per-user with `owner_id = auth.uid()` RLS and an `owner_id` trigger that fills from `auth.uid()` on insert. Both are reached through `SECURITY INVOKER` functions (`match_exercise_kb`, `match_user_memories`, `upsert_user_memory`) so the caller's RLS is always in force, and vectors use an `ivfflat` cosine index.

E. Trade-offs & system design

Q18. Biggest trade-off you made? Making the AI stack — RAG, direct Gemini, and billing — all optional. The upside is the app builds and runs anywhere, which is great for development, web, and tests. The cost is that a fully-featured deployment needs several external services wired up, and I maintain two coach paths (RAG and the 30-day-dump direct path). I think the optionality is worth it for a project meant to be cloned and run, but it's real surface area.

Q19. How would you scale the RAG service past a single box? First, move the rate limiter to a shared store (Redis) so the ceiling isn't multiplied by worker count. Second, if the KB grows, switch the `ivfflat` index to `hnsw` for better recall at scale. The verify-then-spend and JWT-forwarding design already scales horizontally because it's stateless per request — the only stateful piece is the limiter, which is exactly what I'd externalize first.

Q20. How would you reduce false positives in the crisis detector without losing recall? I deliberately tuned it for recall, so I'd be cautious. I'd layer a cheap on-device classifier behind the regex to downgrade obvious non-crisis uses of trigger words, keep the regex as the first gate, and keep the prompt-level safety preamble as the backstop. I would not trade recall for precision here — the asymmetry of the failure modes is the whole point.

Q21. If Gemini changed its embedding model or dimension, what breaks? The 768-dim size is hard-coded in `db/rag.sql`, `app/config.py`, and validated per embedding, so a model change is a schema migration plus a full re-ingest of both stores — the old vectors aren't comparable to new ones. I made that explicit rather than silent precisely so the cost of changing models is visible up front.

F. Behavioral

Q22. What are you most proud of technically? The isolation model and the idempotent offline outbox. Isolation is a database invariant, not a coding convention — the user's JWT flows to Postgres and RLS decides visibility, so a bug in my service can't leak another user's memories, and tests assert exactly that. The outbox turns a lost tap into a durable, replay-once write using the client UUID as the primary key, which is a small idea that quietly solves double-counting.

Q23. What was the hardest part? Getting the data-isolation story airtight and *proving* it. It's easy to say "we forward the JWT"; it's harder to make it a tested contract — that the outbound memory lookup carries the user's bearer token and not the anon/service key, that an unverified token never reaches Gemini, and that a `429` has spent nothing. Writing those tests with mocked network (respx) is what gave me confidence the guarantee is real.

Q24. The repo has heavy process — a build "constitution", agent personas, SPEC-GAPs. Why? ComeBack was built under a strict CLAUDE.md that enforces tests-first, independent review, and an anti-hallucination rule: if it's not in the spec, don't invent it — log a SPEC-GAP and take the most conservative reversible default instead. That's why things like the rate-limiter scope (SPEC-GAP-016) and the exercise-table write policy (SPEC-GAP-018) are documented decisions rather than silent guesses. The discipline kept the security-sensitive parts honest.

Q25. What would you do next? Externalize the rate limiter to Redis, let users curate the coach's long-term memory in-app rather than only via the AI extractor, add a reranker between retrieval and generation, and embed individual chat turns as memories for longer-horizon recall. Product-wise, finish the store listing and the screenshots/demo the README still marks as TBD.

Q26. What did you learn? That the defensible value in an AI feature is usually the boring infrastructure around the model — isolation, verify-before-spend, graceful degradation, idempotency — far more than the prompt itself. The streak and safety logic taught me the same lesson in miniature: the product promise lives in a handful of pure functions I could actually test, not in the UI.

Quick Facts (cheat sheet)

- **What:** Privacy-first habit + workout tracker with an AI coach; iOS, Android, Web via Expo.
- **Thesis:** A lapse is a comeback — relapse-aware streaks never cold-reset to zero.
- **Client:** Expo SDK 54, React Native 0.81, React 19, TypeScript 5.9 strict, expo-router.
- **Data:** Supabase Postgres + owner-only RLS; pgvector for RAG; TanStack Query + Zustand.
- **AI:** Gemini `gemini-2.0-flash` + `text-embedding-004` (768 dims); optional FastAPI RAG service.
- **Isolation:** User JWT forwarded to Postgres; RLS is a DB invariant, not app code.
- **Safety:** On-device crisis regex short-circuits the LLM; prompt-level safety preamble backstop.
- **Headline features:** relapse-aware streaks, idempotent offline outbox, RAG coach with per-user isolation, salted + rate-limited PIN, RevenueCat `pro` paywall.
- **Quality:** Jest + pytest, CodeQL (JS/TS + Python), Dependabot, release-please, Husky hooks.
- **Stack:** MIT licensed; builds and runs with RAG, billing, and AI key all optional.